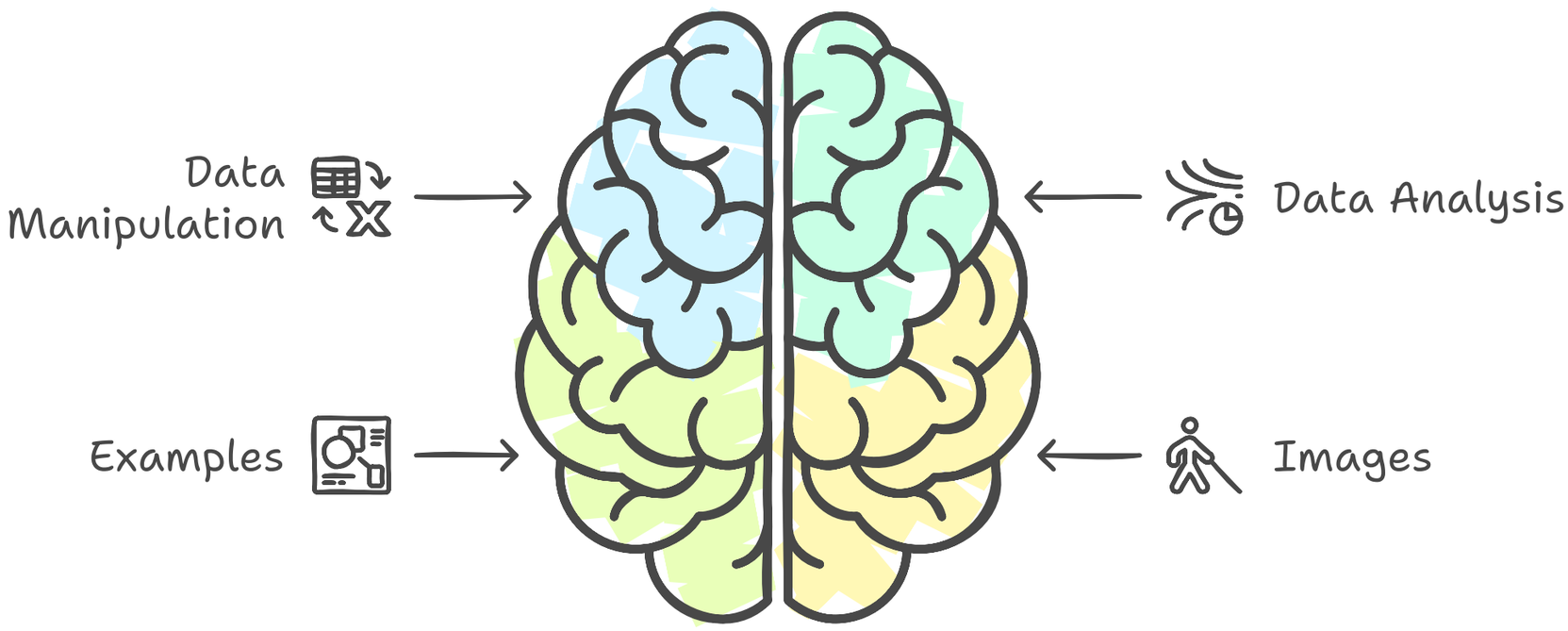
# Most Used PySpark Functions

This document provides a comprehensive list of 50 commonly used PySpark functions, along with examples and images to illustrate their usage. PySpark is a powerful tool for big data processing and analytics, and understanding these functions can significantly enhance your data manipulation and analysis capabilities.




## 1. `show()`

Displays the top rows of a DataFrame.

```
df.show(5)
```

## 2. `select()`

Selects specific columns from a DataFrame.

```
df.select("column1", "column2").show()
```

## 3. `filter()`

Filters rows based on a condition.

```
df.filter(df.age > 21).show()
```

## 4. `groupBy()`

Groups the DataFrame using the specified columns.

```
df.groupBy("column1").count().show()
```

## 5. `agg()`

Aggregates data using specified functions.

```
from pyspark.sql import functions as F
df.groupBy("column1").agg(F.avg("column2")).show()
```

## 6. `withColumn()`

Adds a new column or replaces an existing column.

```
df.withColumn("new_column", df.column1 + 1).show()
```

## 7. `drop()`

Removes specified columns from a DataFrame.

```
df.drop("column1").show()
```

## 8. `orderBy()`

Sorts the DataFrame by specified columns.

```
df.orderBy("column1").show()
```

## 9. `distinct()`

Returns a new DataFrame with distinct rows.

```
df.distinct().show()
```

## 10. `join()`

Joins two DataFrames.

```
df1.join(df2, "column1").show()
```

## 11. `union()`

Combines two DataFrames.

```
df1.union(df2).show()
```

## 12. `limit()`

Limits the number of rows returned.

```
df.limit(10).show()
```

## 13. `describe()`

Generates descriptive statistics.

```
df.describe().show()
```

## 14. `na.fill()`

Fills null values with specified values.

```
df.na.fill(0).show()
```

## 15. `na.drop()`

Drops rows with null values.

```
df.na.drop().show()
```

## 16. `cast()`

Changes the data type of a column.

```
df.withColumn("column1", df.column1.cast("int")).show()
```

## 17. `isNull()`

Checks for null values.

```
df.filter(df.column1.isNull()).show()
```

## 18. `isNotNull()`

Checks for non-null values.

```
df.filter(df.column1.isNotNull()).show()
```

## 19. `when()`

Creates conditional columns.

```
df.withColumn("new_column", F.when(df.column1 > 0,
"Positive").otherwise("Negative")).show()
```

## 20. `concat()`

Concatenates multiple columns.

```
df.withColumn("full_name", F.concat(df.first_name, F.lit(" "),
df.last_name)).show()
```

## 21. `substring()`

Extracts a substring from a column.

```
df.withColumn("sub_column", F.substring(df.column1, 1, 3)).show()
```

## 22. `length()`

Calculates the length of a string column.

```
df.withColumn("length", F.length(df.column1)).show()
```

## 23. `upper()`

Converts a string column to uppercase.

```
df.withColumn("upper_column", F.upper(df.column1)).show()
```

## 24. `lower()`

Converts a string column to lowercase.

```
df.withColumn("lower_column", F.lower(df.column1)).show()
```

## 25. `trim()`

Removes whitespace from both ends of a string.

```
df.withColumn("trimmed_column", F.trim(df.column1)).show()
```

## 26. `date_format()`

Formats a date column.

```
df.withColumn("formatted_date", F.date_format(df.date_column,
"yyyy-MM-dd")).show()
```

## 27. `current_date()`

Returns the current date.

```
df.withColumn("current_date", F.current_date()).show()
```

## 28. `current_timestamp()`

Returns the current timestamp.

```
df.withColumn("current_timestamp", F.current_timestamp()).show()
```

## 29. `year()`

Extracts the year from a date column.

```
df.withColumn("year", F.year(df.date_column)).show()
```

## 30. `month()`

Extracts the month from a date column.

```
df.withColumn("month", F.month(df.date_column)).show()
```

## 31. `dayofmonth()`

Extracts the day of the month from a date column.

```
df.withColumn("day", F.dayofmonth(df.date_column)).show()
```

## 32. `hour()`

Extracts the hour from a timestamp column.

```
df.withColumn("hour", F.hour(df.timestamp_column)).show()
```

## 33. `minute()`

Extracts the minute from a timestamp column.

```
df.withColumn("minute", F.minute(df.timestamp_column)).show()
```

## 34. `second()`

Extracts the second from a timestamp column.

```
df.withColumn("second", F.second(df.timestamp_column)).show()
```

## 35. `datediff()`

Calculates the difference between two dates.

```
df.withColumn("date_diff", F.datediff(df.date_column1, df.date_column2)).show()
```

## 36. `add_months()`

Adds months to a date.

```
df.withColumn("new_date", F.add_months(df.date_column, 1)).show()
```

## 37. `subtract()`

Subtracts two columns.

```
df.withColumn("difference", df.column1 - df.column2).show()
```

## 38. `round()`

Rounds a numeric column.

```
df.withColumn("rounded_column", F.round(df.column1, 2)).show()
```

## 39. `ceil()`

Rounds up to the nearest integer.

```
df.withColumn("ceiled_column", F.ceil(df.column1)).show()
```

## 40. `floor()`

Rounds down to the nearest integer.

```
df.withColumn("floored_column", F.floor(df.column1)).show()
```

## 41. `sum()`

Calculates the sum of a numeric column.

```
df.agg(F.sum("column1")).show()
```

## 42. `avg()`

Calculates the average of a numeric column.

```
df.agg(F.avg("column1")).show()
```

## 43. `min()`

Finds the minimum value in a column.

```
df.agg(F.min("column1")).show()
```

## 44. `max()`

Finds the maximum value in a column.

```
df.agg(F.max("column1")).show()
```

## 45. `count()`

Counts the number of rows.

```
df.agg(F.count("column1")).show()
```

## 46. `corr()`

Calculates the correlation between two columns.

```
df.select(F.corr("column1", "column2")).show()
```

## 47. `covar_pop()`

Calculates the population covariance between two columns.

```
df.select(F.covar_pop("column1", "column2")).show()
```

## 48. `covar_samp()`

Calculates the sample covariance between two columns.

```
df.select(F.covar_samp("column1", "column2")).show()
```

## 49. `collect_list()`

Aggregates values into a list.

```
df.groupBy("column1").agg(F.collect_list("column2")).show()
```

## 50. `collect_set()`

Aggregates values into a set.

```
df.groupBy("column1").agg(F.collect_set("column2")).show()
```

---

This document serves as a quick reference guide for the most commonly used PySpark functions, complete with examples and visual aids. By familiarizing yourself with these functions, you can enhance your data processing and analysis skills in PySpark.